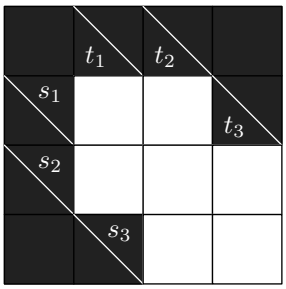
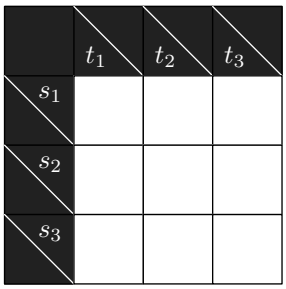
November 2, 2026 at 19:06

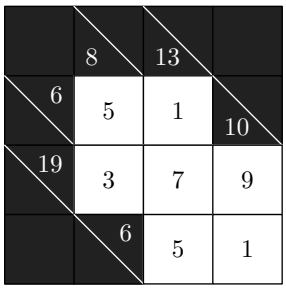
1. Introduction. Exercise 7.2.2.1–432 makes the point that you cannot design a kakuro puzzle by filling the blanks at random and taking the block sums as clues, because almost every set of sums so obtained has more than one solution. It asks for the experiment on two small shapes: a seven-cell staircase and a 3×3 square.



(a)



(b)



6 19 6/8 13 10

Answer 432 reports a long string of numbers—how many fillings, how many of their sum sequences occur only once, how those fall into equivalence classes, which classes are symmetric, which puzzles are easiest and which hardest.

This program checks them. Every count comes out, and so do the names of the symmetric puzzles. Three things do not: the answer's list of possible second rows is one entry short; the count of asymmetric puzzles with no forced move cannot be as large as the answer says; and the puzzle the answer calls the hardest cannot exist, because its across clues and its down clues add up to different totals. The third panel above shows the puzzle I believe was meant.

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strconv"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Types 4 ⟩
⟨ Functions 5 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. ⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "all", "fill␣rgs␣classes␣forced␣cover␣hard␣or␣all")
flag.Parse()
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
if *mode ≡ "fill" ∨ *mode ≡ "all" {
    ⟨ Count the fillings and the sum sequences 8 ⟩
}
if *mode ≡ "rgs" ∨ *mode ≡ "all" {
    ⟨ List the restricted growth strings 11 ⟩
}
if *mode ≡ "classes" ∨ *mode ≡ "all" {
    ⟨ Sort the puzzles into equivalence classes 23 ⟩
}
if *mode ≡ "forced" ∨ *mode ≡ "all" {
    ⟨ Count forced moves and magic blocks 27 ⟩
}
if *mode ≡ "cover" ∨ *mode ≡ "all" {
    ⟨ Solve some kakuro with answer 430(d)'s construction 36 ⟩
}
if *mode ≡ "hard" ∨ *mode ≡ "all" {
    ⟨ Look at the puzzle that cannot exist 38 ⟩
}
```

This code is used in section 1.

4. The two diagrams. A diagram is nothing but its blocks: which cells lie in each horizontal block and which in each vertical one. Cells are numbered in reading order, so the staircase of part (a) has cells 0 and 1 in its top row, 2, 3 and 4 in the middle, 5 and 6 at the bottom, while part (b) is a plain 3×3.

⟨ Types 4 ⟩ ≡

```
type shape struct {
    n int
    rows, cols [][]int
}
```

This code is used in section 1.

5. ⟨ Functions 5 ⟩ ≡

```
func shapes()(a, b, c shape) {
    a = shape{7,
        [][]int{{0, 1}, {2, 3, 4}, {5, 6}},
        [][]int{{0, 2}, {1, 3, 5}, {4, 6}}}
    b = shape{9,
        [][]int{{0, 1, 2}, {3, 4, 5}, {6, 7, 8}},
        [][]int{{0, 3, 6}, {1, 4, 7}, {2, 5, 8}}}
    c = shape{16,   // the four by four of the closing note
        [][]int{{0, 1, 2, 3}, {4, 5, 6, 7}, {8, 9, 10, 11}, {12, 13, 14, 15}},
        [][]int{{0, 4, 8, 12}, {1, 5, 9, 13}, {2, 6, 10, 14}, {3, 7, 11, 15}}}
    return
}
// blockOf says which horizontal and which vertical block each cell is in.
func (s shape) blockOf()(r, c []int) {
    r = make([]int, s.n)
    c = make([]int, s.n)
    for b, blk := range s.rows {
        for _, i := range blk {
            r[i] = b
        }
    }
    for b, blk := range s.cols {
        for _, i := range blk {
            c[i] = b
        }
    }
    return
}
```

This code is also defined in sections 6, 7, 10, 13, 15, 18, 20, 24, 25, 30, 34, 35.

This code is used in section 1.

6. Filling the blanks means writing a digit in every cell with no digit repeated in any block. Walking them all is a plain backtrack over the cells in order, with a bitmap of the digits each block has already used.

⟨ Functions 5 ⟩ +≡

```
func (s shape) fill(visit func(d []int)) {
    rb, cb := s.blockOf()
    d := make([]int, s.n)
    rmask := make([]int, len(s.rows))
    cmask := make([]int, len(s.cols))
    var rec func(i int)
    rec = func(i int) {
        if i ≡ s.n {
            visit(d)
            return
        }
        for x := 1; x ≤ 9; x++ {
            bit := 1 ≪ x
            if rmask[rb[i]] & bit ≠ 0 ∨ cmask[cb[i]] & bit ≠ 0 {
                continue
            }
            rmask[rb[i]] |= bit
            cmask[cb[i]] |= bit
            d[i] = x
            rec(i + 1)
            rmask[rb[i]] ⊘= bit
            cmask[cb[i]] ⊘= bit
        }
    }
    rec(0)
}
```

7. The clues of a puzzle are the six block sums, the horizontal ones first; answer 432 writes them $s_1 s_2 s_3 / t_1 t_2 t_3$. Packing each into five bits makes a sequence into a number that can be counted in a map.

⟨ Functions 5 ⟩ +≡

```
func (s shape) sums(d []int) []int {
    out := make([]int, 0, len(s.rows) + len(s.cols))
    for _, blk := range s.rows {
        t := 0
        for _, i := range blk {
            t += d[i]
        }
        out = append(out, t)
    }
    for _, blk := range s.cols {
        t := 0
        for _, i := range blk {
            t += d[i]
        }
        out = append(out, t)
    }
    return out
}
func code(v []int) int {
    c := 0
    for _, x := range v {
        c = c << 5 | x
    }
    return c
}
func show(v []int) string {
    var w []string
    for i, x := range v {
        if i * 2 == len(v) {
            w = append(w, "/")
        }
        w = append(w, strconv.Itoa(x))
    }
    return strings.Join(w, "␣")
}
```

8. Counting the fillings. The first thing to check is the number of ways to fill the blanks, and the second is how few of those are reconstructible from their sums. Both diagrams come out exactly as answer 432 says: 1432872 fillings for (a), of which 78690 sum sequences occur once and no more, and 43038576 for (b), of which 6840 do.

⟨ Count the fillings and the sum sequences 8 ⟩ ≡

```
a, b, _ := shapes()
for _, s := range []shape { a, b }{
    total := 0
    seen := map[int]int{}
    s.fill(func(d []int) {
        total++
        seen[code(s.sums(d))]++
    })
    once := 0
    for _, n := range seen {
        if n ≡ 1 {
            once++
        }
    }
    fmt.Printf("%d␣cells:␣%d␣fillings,␣%d␣distinct␣sum␣sequences," +
        "␣%d␣of␣them␣unique␣(%.3f%%)\n",
        s.n, total, len(seen), once, 100 * float64(once)/float64(total))
    ⟨ Break the fillings down by how many digits they use 9 ⟩
}
```

This code is used in section 3.

9. Answer 432 does the counting by patterns rather than by digits: a filling is described by which cells match, and then the matching classes are given actual digits in $9^{\underline{k}}$ ways when there are k of them. So the number of fillings using exactly k digits, divided by the falling factorial, is the number of patterns—the "restricted growth strings" of the answer.

⟨ Break the fillings down by how many digits they use 9 ⟩ ≡

```
byDigits := map[int]int{}
s.fill(func(d []int) {
    m := 0
    for _, x := range d {
        m |= 1 ≪ x
    }
    byDigits[bitsOf(m)]++
})
for k := 2; k ≤ 9; k++ {
    if byDigits[k] > 0 {
        fmt.Printf("␣␣␣%d␣digits:␣%9d␣fillings␣=␣%5d␣patterns␣times␣%d\n",
            k, byDigits[k], byDigits[k]/falling(9, k), falling(9, k))
    }
}
```

This code is used in section 8.

10. ⟨ Functions 5 ⟩ +≡

```
func bitsOf(x int) int {
    n := 0
    for; x ≠ 0; x &= x − 1 {
        n++
    }
    return n
}
func falling(n, k int) int {
    p := 1
    for i := range k {
        p *= n − i
    }
    return p
}
```

11. The patterns themselves. Answer 432 sketches the patterns for diagram (a) directly: "we can assume that the top row is '12'; then the second row is either '213' or '234' or '312' or '314' or '34x' for $1 \le x \le 5$." Since the three cells of the second row lie in one block they must differ, so '343' and '344' are out and that list holds seven strings. There are eight: '231' is missing.

⟨ List the restricted growth strings 11 ⟩ ≡

```
a, _, _ := shapes()
⟨ Grow every restricted growth string of diagram (a) 12 ⟩
fmt.Printf("diagram␣(a):␣%d␣patterns", len(pat))
byMax := map[int]int{}
for _, p := range pat {
    byMax[p[maxAt(p)]]++
}
for k := 2; k ≤ 7; k++ {
    if byMax[k] > 0 {
        fmt.Printf("␣␣%d:%d", k, byMax[k])
    }
}
fmt.Println()
⟨ Report the second rows that can occur 14 ⟩
```

This code is used in section 3.

12. A pattern is built cell by cell: a cell may join any class already used, so long as no cell of its two blocks is in that class, or start the next new one.

⟨ Grow every restricted growth string of diagram (a) 12 ⟩ ≡

```
rb, cb := a.blockOf()
var pat [][]int
p := make([]int, 0, a.n)
var grow func()
grow = func() {
    i := len(p)
    if i ≡ a.n {
        pat = append(pat, append([]int{}, p...))
        return
    }
    top := 1
    for _, v := range p {
        if v + 1 > top {
            top = v + 1
        }
    }
    for v := 1; v ≤ top; v++ {
        ok := true
        for j := range i {
            if p[j] ≡ v ∧ (rb[j] ≡ rb[i] ∨ cb[j] ≡ cb[i]) {
                ok = false
            }
        }
        if ok {
            p = append(p, v)
            grow()
            p = p[:len(p) − 1]
        }
    }
}
grow()
```

This code is used in section 11.

13. ⟨ Functions 5 ⟩ +≡

```
func maxAt(p []int) int {
    k := 0
    for i, v := range p {
        if v > p[k] {
            k = i
        }
        _ = v
    }
    return k
}
```

14. ⟨ Report the second rows that can occur 14 ⟩ ≡

```
rows := map[string]bool{}
for _, q := range pat {
    if q[0] == 1 ∧ q[1] == 2 {
        rows[fmt.Sprintf("%d%d%d", q[2], q[3], q[4])] = true
    }
}
var list []string
for r := range rows {
    list = append(list, r)
}
sort.Strings(list)
fmt.Printf("␣␣␣with␣the␣top␣row␣12,␣the␣second␣row␣can␣be:␣%s\n",
    strings.Join(list, "␣"))
fmt.Println("␣␣␣answer␣432␣lists␣213,␣234,␣312,␣314␣and␣34x␣for␣1␣<=␣x␣<=␣5")
```

This code is used in section 11.

15. Equivalent puzzles. Two clue sequences describe the same puzzle when a symmetry of the diagram carries one to the other. Every diagram here has the *dual*, which replaces each clue s for a block of length k by $10k - s$ and each digit d by $10 - d$, and the *transpose*, which swaps the horizontal clues with the vertical ones. Diagram (a) also has the half turn, which reverses each triple; diagram (b), being a square, lets its rows and its columns be permuted at will. That makes a group of order 8 for (a) and of order 144 for (b).

⟨ Functions 5 ⟩ +≡

```
func (s shape) symmetries() []func([]int) []int {
    var k []int
    for _, b := range s.rows {
        k = append(k, len(b))
    }
    for _, b := range s.cols {
        k = append(k, len(b))
    }
    g := []func([]int) []int {
        func(v []int) []int {
            w := make([]int, len(v))
            for i := range v {
                w[i] = 10 * k[i] - v[i]
            }
            return w
        },
        func(v []int) []int {
            return []int{v[3], v[4], v[5], v[0], v[1], v[2]}
        },
    }
    ⟨ Add the symmetries peculiar to each diagram 16 ⟩
    return g
}
```

16. The staircase has the half turn, which reverses both triples.

⟨ Add the symmetries peculiar to each diagram 16 ⟩ ≡

```
if s.n ≡ 7 {
    g = append(g, func(v []int) []int {
        return []int{v[2], v[1], v[0], v[5], v[4], v[3]}
    })
}
```

This code is also defined in section 17.

This code is used in section 15.

17. The square has more: two transpositions generate all six ways to permute the rows, and two more all six ways to permute the columns.

⟨ Add the symmetries peculiar to each diagram 16 ⟩ +≡

```
if s.n ≡ 9 {
    for _, p := range [][3]int {{1, 0, 2}, {0, 2, 1} }{
        p := p
        g = append(g, func(v []int) []int {
            return []int{v[p[0]], v[p[1]], v[p[2]], v[3], v[4], v[5]}
        })
        g = append(g, func(v []int) []int {
            return []int{v[0], v[1], v[2], v[3 + p[0]], v[3 + p[1]], v[3 + p[2]]}
        })
    }
}
```

18. ⟨ Functions 5 ⟩ +≡

```
func (s shape) orbit(v []int) map[int][]int {
    gens := s.symmetries()
    out := map[int][]int{code(v): v}
    for again := true; again; {
        again = false
        for _, w := range out {
            for _, g := range gens {
                u := g(w)
                if _, ok := out[code(u)]; ¬ok {
                    out[code(u)] = u
                    again = true
                }
            }
        }
    }
    return out
}
// puzzles lists the clue sequences that only one filling produces. One sweep
// counts the sequences and a second picks up those that came out once.
func (s shape) puzzles() [][]int {
    seen := map[int]int{}
    s.fill(func(d []int) { seen[code(s.sums(d))]++ })
    var out [][]int
    done := map[int]bool{}
    ⟨ Gather the sequences that were seen only once 19 ⟩
    return out
}
```

19. ⟨ Gather the sequences that were seen only once 19 ⟩ ≡

```
s.fill(func(d []int) {
    c := code(s.sums(d))
    if seen[c] ≡ 1 ∧ ¬done[c] {
        done[c] = true
        out = append(out, s.sums(d))
    }
})
```

This code is used in section 18.

20. ⟨ Functions 5 ⟩ +≡

```
// classify groups the puzzles, returning one representative of each class and
// the order of its stabilizer – the number of symmetries it has. The order
// of the whole group is the largest orbit seen, since some puzzle has no
// symmetry at all.
func (s shape) classify()(reps [][]int, stab []int) {
    seen := map[int]bool{}
    var size []int
    g := 1
    for _, v := range s.puzzles() {
        if seen[code(v)] {
            continue
        }
        ⟨ Take the least member of this orbit as its name 21 ⟩
    }
    for _, n := range size {
        stab = append(stab, g/n)
    }
    return
}
```

21. ⟨ Take the least member of this orbit as its name 21 ⟩ ≡

```
orb := s.orbit(v)
best := v
for _, w := range orb {
    if code(w) < code(best) {
        best = w
    }
}
for c := range orb {
    seen[c] = true
}
reps = append(reps, best)
size = append(size, len(orb))
if len(orb) > g {
    g = len(orb)
}
```

This code is used in section 20.

22. There are 190 classes of diagram (a) with a symmetry, too many to print, so only the small families get named—which is what answer 432 names too.

⟨ Name the symmetric puzzles, when there are few enough to name 22 ⟩ ≡

```
for m := 2; m ≤ 144; m++ {
    if byStab[m] ≡ 0 ∨ byStab[m] > 5 {
        continue
    }
    for i, v := range reps {
        if stab[i] ≡ m {
            fmt.Printf("␣␣␣␣␣␣␣␣%s\n", show(v))
        }
    }
}
```

This code is used in section 23.

23. Answer 432 says diagram (a) has 9932 classes, one of which has four symmetries and 190 of which have two, and that diagram (b) has 49, of which three are symmetric. All of that comes out, and so do the names it gives.

⟨ Sort the puzzles into equivalence classes 23 ⟩ ≡

```
a, b, _ := shapes()
for _, s := range []shape{ a, b }{
    reps, stab := s.classify()
    byStab := map[int]int{}
    for _, m := range stab {
        byStab[m]++
    }
    fmt.Printf("%d␣cells:␣%d␣equivalence␣classes\n", s.n, len(reps))
    for m := 1; m ≤ 144; m++ {
        if byStab[m] > 0 {
            fmt.Printf("␣␣␣%5d␣with␣%d␣symmetr%s\n", byStab[m], m,
                map[bool]string{true: "y", false: "ies"}[m ≡ 1])
        }
    }
    ⟨ Name the symmetric puzzles, when there are few enough to name 22 ⟩
}
```

This code is used in section 3.

24. Forced moves and magic blocks. Answer 430(b) calls a block "magic" when its clue admits only one combination of digits. Answer 432 speaks of "forced moves" without saying what they are, but its three examples pin the meaning down. A solver looking at a fresh diagram can fill in a cell at once in two ways: the two blocks through the cell may agree on only one digit—call it a naked single—or a magic block may have a digit that fits in only one of its cells, a hidden single. Counting both gives one forced move for the puzzle 5 19 6/6 10 14 of exercise 430, "in the lower right corner" as the answer says, and four apiece for 4 15 12/12 15 4 and 4 15 16/12 15 8, which is what the answer says of them.

⟨ Functions 5 ⟩ +≡

```
// combos lists as bitmaps the k-element sets of digits that sum to n.
func combos(n, k int) []int {
    var out []int
    for x := 1; x < 512; x++ {
        c, t := 0, 0
        for d := 1; d ≤ 9; d++ {
            if x & (1 ≪ (d − 1)) ≠ 0 {
                c++
                t += d
            }
        }
        if c ≡ k ∧ t ≡ n {
            out = append(out, x ≪ 1)
        }
    }
    return out
}
// reach is the set of digits an n-in-k block can hold, and a block is magic
// when it has just one combination.
func reach(n, k int)(mask int, magic bool) {
    c := combos(n, k)
    for _, x := range c {
        mask |= x
    }
    return mask, len(c) ≡ 1
}
```

25. ⟨ Functions 5 ⟩ +≡

```
func (s shape) forced(v []int)(moves, magics int) {
    rb, cb := s.blockOf()
    rm := make([]int, len(s.rows))
    cm := make([]int, len(s.cols))
    for b := range s.rows {
        var mg bool
        rm[b], mg = reach(v[b], len(s.rows[b]))
        if mg {
            magics++
        }
    }
    for b := range s.cols {
        var mg bool
        cm[b], mg = reach(v[len(s.rows) + b], len(s.cols[b]))
        if mg {
            magics++
        }
    }
    settled := make([]bool, s.n)
    for i := range s.n {
        if bitsOf(rm[rb[i]] & cm[cb[i]]) ≡ 1 {
            settled[i] = true
        }
    }
    ⟨ Let every magic block place the digits that have nowhere else to go 26 ⟩
    for i := range s.n {
        if settled[i] {
            moves++
        }
    }
    return
}
```

26. ⟨ Let every magic block place the digits that have nowhere else to go 26 ⟩ ≡

```
hidden := func(blocks [][]int, off int, cross []int, crossMask []int) {
    for b, blk := range blocks {
        c := combos(v[off + b], len(blk))
        if len(c) ≠ 1 {
            continue
        }
        for x := 1; x ≤ 9; x++ {
            if c[0] & (1 ≪ x) ≡ 0 {
                continue
            }
            where, n := −1, 0
            for _, i := range blk {
                if crossMask[cross[i]] & (1 ≪ x) ≠ 0 {
                    where, n = i, n + 1
                }
            }
            if n ≡ 1 {
                settled[where] = true
            }
        }
    }
}
hidden(s.rows, 0, cb, cm)
hidden(s.cols, len(s.rows), rb, rm)
```

This code is used in section 25.

27. With that reading, the three puzzles answer 432 names come out right; but the counts that follow do not. The answer says 4011 of the asymmetric puzzles of diagram (a) have no forced move, and that 570 of those have no magic block. Here the numbers are 3172 and 576. And 4011 is too large to be right whatever a forced move is taken to be, so long as a naked single is one: only 3360 of the 9741 asymmetric puzzles have no naked single at all. The answer's own example suggests that a naked single does count, since 5 19 6/6 10 14 has no magic block and so the forced move in its lower right corner can only be a naked single. But one example suggests a definition; it does not settle one.

⟨ Count forced moves and magic blocks 27 ⟩ ≡

```
a, b, _ := shapes()
for _, v := range [][]int {{5, 19, 6, 6, 10, 14}, {4, 15, 12, 12, 15, 4},
    {4, 15, 16, 12, 15, 8} }{
        m, g := a.forced(v)
        fmt.Printf("%s: %d forced move(s), %d magic block(s)\n", show(v), m, g)
    }
for _, s := range []shape { a, b }{
    reps, stab := s.classify()
    none, noneNoMagic, noNaked := 0, 0, 0
    rootFree, rootFreeNoMagic := 0, 0
    for i, v := range reps {
        if stab[i] ≠ 1 {
            continue
        }
        m, g := s.forced(v)
        if m ≡ 0 {
            none++
            if g ≡ 0 {
                noneNoMagic++
            }
        }
        ⟨ Count the asymmetric puzzles with no naked single 28 ⟩
        ⟨ Count the asymmetric puzzles with no one-option item at the root 29 ⟩
    }
    fmt.Printf("%d cells: of the asymmetric puzzles, %d have no forced move," +
        " %d of those no magic block; %d have no naked single\n",
        s.n, none, noneNoMagic, noNaked)
    fmt.Printf("     %d have no item with a single option at the root of" +
        " answer 430(d)'s problem, %d of those no magic block\n",
        rootFree, rootFreeNoMagic)
}
```

This code is used in section 3.

28. ⟨ Count the asymmetric puzzles with no naked single 28 ⟩ ≡

```
rb, cb := s.blockOf()
naked := 0
for j := range s.n {
    rm, _ := reach(v[rb[j]], len(s.rows[rb[j]]))
    cm, _ := reach(v[len(s.rows) + cb[j]], len(s.cols[cb[j]]))
    if bitsOf(rm & cm) ≡ 1 {
        naked++
    }
}
if naked ≡ 0 {
    noNaked++
}
```

This code is used in section 27.

29. The argument above leans on a single example for the claim that a naked single is a forced move, and that is thinner than it looked when I first wrote it. I came back to it in November 2026, after a bug in another of these readings (exercise 334) had hidden behind an assumption that all its checks shared. Knuth measures difficulty with Algorithm C, so a natural alternative is that a forced move is a primary item with only one option at the root of answer 430(d)'s problem. That is not the same thing: in that construction a cell has one option for every pair of combinations its two blocks might use, so a naked single can still leave a cell with several options. Under this reading the counts are 2750 and 470—no closer to 4011 and 570 than the others.

⟨ Count the asymmetric puzzles with no one-option item at the root 29 ⟩ ≡

```
{
    X, Y := s.combsFor(v)
    lines := strings.Split(strings.TrimSpace(s.cover(X, Y)), "\n")
    head, _, _ := strings.Cut(lines[0], "|")
    opts := map[string]int{}
    for _, it := range strings.Fields(head) {
        opts[it] = 0
    }
    for _, l := range lines[1:] {
        for _, it := range strings.Fields(l) {
            if _, ok := opts[it]; ok {
                opts[it]++
            }
        }
    }
    single := false
    for _, k := range opts {
        if k == 1 {
            single = true
        }
    }
    if ¬single {
        rootFree++
        if g == 0 {
            rootFreeNoMagic++
        }
    }
}
```

This code is used in section 27.

30. Answer 430(d)'s exact cover problem. Answer 432 measures difficulty by the size of Algorithm C's search tree, "using the construction of answer 430(d)." That construction is worth having here in any case. It gives a primary item ij for every cell; a primary item H_{hpx} for every digit x of every combination X_{hp} of every horizontal block h, and V_{vqy} likewise; and a secondary item H_h per block, whose colour says which combination that block will use. A cell's options say which digit it takes and which combinations its two blocks are using, and a further option for each unused combination absorbs the items it left behind.

```
⟨ Functions 5 ⟩ +≡
    func (s shape) cover(X, Y [][][]int) string {
        rb, cb := s.blockOf()
        var prim, sec, opts []string
        for i := range s.n {
            prim = append(prim, fmt.Sprintf("c%d", i))
        }
        ⟨ Name an item for every element of every combination 31 ⟩
        ⟨ Write an option for every digit a cell can take 32 ⟩
        ⟨ Write an option to absorb each combination that goes unused 33 ⟩
        return strings.Join(prim, "␣") + "␣|␣" + strings.Join(sec, "␣") + "\n" +
            strings.Join(opts, "\n") + "\n"
    }
```

31. ⟨ Name an item for every element of every combination 31 ⟩ ≡

```
    for b := range X {
        for p := range X[b] {
            for _, x := range X[b][p] {
                prim = append(prim, fmt.Sprintf("H%d.%d.%d", b, p, x))
            }
        }
        sec = append(sec, fmt.Sprintf("h%d", b))
    }
    for b := range Y {
        for q := range Y[b] {
            for _, y := range Y[b][q] {
                prim = append(prim, fmt.Sprintf("V%d.%d.%d", b, q, y))
            }
        }
        sec = append(sec, fmt.Sprintf("v%d", b))
    }
```

This code is used in section 30.

32. ⟨ Write an option for every digit a cell can take 32 ⟩ ≡

```
for i := range s.n {
    h, v := rb[i], cb[i]
    for p := range X[h] {
        for q := range Y[v] {
            for _, x := range X[h][p] {
                if ¬inSet(Y[v][q], x) {
                    continue
                }
                opts = append(opts, fmt.Sprintf(
                    "c%d␣H%d.%d.%d␣h%d:%d␣V%d.%d.%d␣v%d:%d",
                    i, h, p, x, h, p, v, q, x, v, q))
            }
        }
    }
}
```

This code is used in section 30.

33. ⟨ Write an option to absorb each combination that goes unused 33 ⟩ ≡

```
absorb := func(Z [][][]int, tag string) {
    for b := range Z {
        for p := range Z[b] {
            for pp := range Z[b] {
                if p ≡ pp {
                    continue
                }
                var w []string
                for _, x := range Z[b][p] {
                    w = append(w, fmt.Sprintf("%s%d.%d.%d",
                        strings.ToUpper(tag), b, p, x))
                }
                opts = append(opts, strings.Join(w, "␣") +
                    fmt.Sprintf("␣%s%d:%d", tag, b, pp))
            }
        }
    }
}
absorb(X, "h")
absorb(Y, "v")
```

This code is used in section 30.

34. ⟨ Functions 5 ⟩ +≡

```
func inSet(a []int, x int) bool {
    for _, v := range a {
        if v ≡ x {
            return true
        }
    }
    return false
}
// digitsOf turns a bitmap of digits into a list.
func digitsOf(m int) []int {
    var out []int
    for x := 1; x ≤ 9; x++ {
        if m & (1 ≪ x) ≠ 0 {
            out = append(out, x)
        }
    }
    return out
}
// combsFor is the list of combinations of every block of a puzzle.
func (s shape) combsFor(v []int)(X, Y [][][]int) {
    X = make([][][]int, len(s.rows))
    Y = make([][][]int, len(s.cols))
    for b := range s.rows {
        for _, m := range combos(v[b], len(s.rows[b])) {
            X[b] = append(X[b], digitsOf(m))
        }
    }
    for b := range s.cols {
        for _, m := range combos(v[len(s.rows) + b], len(s.cols[b])) {
            Y[b] = append(Y[b], digitsOf(m))
        }
    }
    return
}
```

35. ⟨ Functions 5 ⟩ +≡

```
func (s shape) solve(X, Y [][][]int)(count int, first []int, nodes uint64) {
    x := cells.NewXCC()
    res := x.Dance(strings.NewReader(s.cover(X, Y)))
    for sol := range res.Solutions {
        count++
        if first ≠ Λ {
            continue
        }
        first = make([]int, s.n)
        for _, opt := range sol {
            if ¬strings.HasPrefix(opt[0], "c") {
                continue
            }
            i, _ := strconv.Atoi(opt[0][1:])
            d, _ := strconv.Atoi(strings.Split(opt[1], ".")[2])
            first[i] = d
        }
    }
    return count, first, x.Nodes()
}
```

36. Two puzzles from exercise 430 test the construction. Its mini-kakuro is the staircase with clues 5 19 6/6 10 14, and answer 430(a) says the lower right corner must be 5; the generalized kakuro of 430(c), where the blocks are given lists of combinations rather than sums, must have 7 9 8 down the middle.

⟨ Solve some kakuro with answer 430(d)'s construction 36 ⟩ ≡

```
a, _, _ := shapes()
mini := []int{5, 19, 6, 6, 10, 14}
X, Y := a.combsFor(mini)
n, d, nodes := a.solve(X, Y)
fmt.Printf("%s: %d solution%v, lower right %d, %d nodes\n",
    show(mini), n, d, d[a.n − 1], nodes)
two := [][]int{{1, 3}, {3, 5}, {5, 7}}
three := [][]int{{1, 3, 5}, {1, 7, 9}, {2, 4, 6}, {6, 8, 9}, {7, 8, 9}}
even := [][]int{{2, 4}, {4, 6}, {6, 8}}
gx := [][][]int{two, three, even}
n, d, nodes = a.solve(gx, gx)
fmt.Printf("exercise 430(c): %d solution%v, middle row %d%d%d, %d nodes\n",
    n, d, d[2], d[3], d[4], nodes)
⟨ Try the four by four of the closing note 37 ⟩
```

This code is used in section 3.

37. The note that closes answer 432 says that a kakuro whose blanks make a 4×4 grid is very hard to come by, and that 11 15 23 29/12 15 23 28 is one, with a search tree of 488 nodes. It does have a single solution. The tree here is smaller, but that is a fact about this program rather than about the puzzle: two implementations of Algorithm C that break ties differently walk different trees.

⟨ Try the four by four of the closing note 37 ⟩ ≡

```
_, _, c := shapes()
big := []int{11, 15, 23, 29, 12, 15, 23, 28}
X, Y = c.combsFor(big)
n, d, nodes = c.solve(X, Y)
fmt.Printf("%s: %d solution %v, %d nodes\n", show(big), n, d, nodes)
```

This code is used in section 36.

38. The puzzle that cannot exist. Answer 432 ends its first part by naming the hardest puzzle of diagram (a) among the asymmetric ones with no forced move and no magic block: "puzzle 6 19 6/8 11 10 is the hardest, in the sense that it maximizes the number of nodes (79) in Algorithm C's search tree." But 6 19 6/8 11 10 is not a puzzle at all. Its three across clues add to 31 and its three down clues add to 29, and both of those totals are the sum of the same seven digits, so they have to agree.

Changing one clue can repair it in six ways, and only one of them lands where the answer says it should: 6 19 6/8 13 10 has a unique solution, no symmetry, no forced move and no magic block. That must be the puzzle meant.

⟨ Look at the puzzle that cannot exist 38 ⟩ ≡

```
a, _, _ := shapes()
printed := []int{6, 19, 6, 8, 11, 10}
fmt.Printf("%s: across %d, down %d\n", show(printed),
    printed[0] + printed[1] + printed[2], printed[3] + printed[4] + printed[5])
⟨ Try every repair that changes one clue 39 ⟩
```

This code is used in section 3.

39. ⟨ Try every repair that changes one clue 39 ⟩ ≡

```
for i := range 6 {
    lo, hi := 3, 17
    if i ≡ 1 ∨ i ≡ 4 {
        lo, hi = 6, 24
    }
    for x := lo; x ≤ hi; x++ {
        v := append([]int{}, printed...)
        v[i] = x
        if x ≡ printed[i] ∨ v[0] + v[1] + v[2] ≠ v[3] + v[4] + v[5] {
            continue
        }
        X, Y := a.combsFor(v)
        n, d, nodes := a.solve(X, Y)
        if n ≠ 1 {
            fmt.Printf("   %s: %d solutions\n", show(v), n)
            continue
        }
        m, g := a.forced(v)
        fmt.Printf("   %s: unique %v, %d nodes, %d symmetr(y/ies)," +
            " %d forced, %d magic\n",
            show(v), d, nodes, 8/len(a.orbit(v)), m, g)
    }
}
```

This code is used in section 38.

40. Index.

a: 5, 8, 11, 12, 23, 27, 34, 36, 38, 39.
absorb: 33.
again: 18.
append: 7, 12, 14, 15, 16, 17, 19, 20, 21, 24, 30, 31, 32, 33, 34, 39.
Atoi: 35.
b: 5, 8, 15, 23, 25, 26, 27, 31, 33, 34.
best: 21.
big: 37.
bit: 6.
bitsOf: 9, 10, 25, 28.
blk: 5, 7, 26.
blockOf: 5, 6, 12, 25, 28, 30.
blocks: 26.
byDigits: 9.
byMax: 11.
byStab: 22, 23.
c: 5, 7, 19, 21, 24, 26, 37.
cb: 6, 12, 25, 26, 28, 30, 32.
cells: 1, 35.
classify: 20, 23, 27.
cm: 25, 26, 28.
cmask: 6.
code: 7, 8, 18, 19, 20, 21.
cols: 4, 5, 6, 7, 15, 25, 26, 28, 34.
combos: 24, 26, 34.
combsFor: 29, 34, 36, 37, 39.
count: 35.
cover: 29, 30, 35.
cross: 26.
crossMask: 26.
Cut: 29.
d: 6, 7, 8, 9, 18, 19, 24, 35, 36, 37, 39.
Dance: 35.
digitsOf: 34.
done: 18, 19.
even: 36.
falling: 9, 10.
Fields: 29.
fill: 6, 8, 9, 18, 19.
first: 35.
flag: 2.
fmt: 8, 9, 11, 14, 22, 23, 27, 30, 31, 32, 33, 36, 37, 38, 39.
forced: 25, 27, 39.
g: 15, 16, 17, 18, 20, 21, 27, 29, 39.
gens: 18.
grow: 12.
gx: 36.
h: 32.
HasPrefix: 35.
head: 29.
hi: 39.
hidden: 26.
i: 5, 6, 7, 10, 12, 13, 15, 22, 25, 26, 27, 30, 32, 35, 39.
ij: 30.
inSet: 32, 34.
it: 29.
Itoa: 7.
j: 12, 28.
Join: 7, 14, 30, 33.
k: 9, 10, 11, 13, 15, 24, 29.
l: 29.
len: 6, 7, 8, 11, 12, 15, 21, 23, 24, 25, 26, 28, 34, 39.
lines: 29.
list: 14.
lo: 39.
m: 9, 22, 23, 27, 34, 39.
magic: 24.
magics: 25.
main: 1.
make: 5, 6, 7, 12, 15, 25, 34, 35.
mask: 24.
maxAt: 11, 13.
mg: 25.
mini: 36.
mode: 2, 3.
moves: 25.
n: 4, 5, 6, 8, 10, 12, 16, 17, 20, 23, 24, 25, 26, 27, 28, 30, 32, 35, 36, 37, 39.
naked: 28.
NewReader: 35.
NewXCC: 35.
Nodes: 35.
nodes: 35, 36, 37, 39.
noNaked: 27, 28.
none: 27.
noneNoMagic: 27.
off: 26.
ok: 12, 18, 29.
once: 8.
opt: 35.

opts: 29, 30, 32, 33.
orb: 21.
orbit: 18, 21, 39.
out: 7, 18, 19, 24, 34.
p: 10, 11, 12, 13, 17, 31, 32, 33.
Parse: 2.
pat: 11, 12, 14.
pp: 33.
prim: 30, 31.
printed: 38, 39.
Printf: 8, 9, 11, 14, 22, 23, 27, 36, 37, 38, 39.
Println: 11, 14.
puzzles: 18, 20.
q: 14, 31, 32.
r: 5, 14.
rb: 6, 12, 25, 26, 28, 30, 32.
reach: 24, 25, 28.
rec: 6.
reps: 20, 21, 22, 23, 27.
res: 35.
rm: 25, 26, 28.
rmask: 6.
rootFree: 27, 29.
rootFreeNoMagic: 27, 29.
rows: 4, 5, 6, 7, 14, 15, 25, 26, 28, 34.
s: 5, 6, 7, 8, 9, 15, 16, 17, 18, 19, 20, 21, 23, 25, 26, 27, 28, 29, 30, 32, 34, 35.
sec: 30, 31.
seen: 8, 18, 19, 20, 21.
settled: 25, 26.
shape: 4, 5, 6, 7, 8, 15, 18, 20, 23, 25, 27, 30, 34, 35.
shapes: 5, 8, 11, 23, 27, 36, 37, 38.
show: 7, 22, 27, 36, 37, 38, 39.
single: 29.
size: 20, 21.
sol: 35.
Solutions: 35.
solve: 35, 36, 37, 39.
sort: 14.
Split: 29, 35.
Sprintf: 14, 30, 31, 32, 33.
stab: 20, 22, 23, 27.
strconv: 7, 35.
String: 2.
Strings: 14.
strings: 7, 14, 29, 30, 33, 35.
sums: 7, 8, 18, 19.
symmetries: 15, 18.
t: 7, 24.
tag: 33.
three: 36.
top: 12.
total: 8.
ToUpper: 33.
TrimSpace: 29.
two: 36.
u: 18.
v: 7, 12, 13, 15, 16, 17, 18, 20, 21, 22, 25, 26, 27, 28, 29, 32, 34, 39.
visit: 6.
w: 7, 15, 18, 21, 33.
where: 26.
X: 29, 30, 31, 32, 33, 34, 35, 36, 37, 39.
x: 6, 7, 9, 10, 24, 26, 31, 32, 33, 34, 35, 39.
Y: 29, 30, 31, 32, 33, 34, 35, 36, 37, 39.
y: 31.
Z: 33.

⟨ Add the symmetries peculiar to each diagram 16 ⟩ Used in section 15
⟨ Break the fillings down by how many digits they use 9 ⟩ Used in section 8
⟨ Count forced moves and magic blocks 27 ⟩ Used in section 3
⟨ Count the asymmetric puzzles with no naked single 28 ⟩ Used in section 27
⟨ Count the asymmetric puzzles with no one-option item at the root 29 ⟩ Used in section 27
⟨ Count the fillings and the sum sequences 8 ⟩ Used in section 3
⟨ Do what the mode asks 3 ⟩ Used in section 1
⟨ Functions 5 ⟩ Used in section 1
⟨ Gather the sequences that were seen only once 19 ⟩ Used in section 18
⟨ Grow every restricted growth string of diagram (a) 12 ⟩ Used in section 11
⟨ Let every magic block place the digits that have nowhere else to go 26 ⟩ Used in section 25
⟨ List the restricted growth strings 11 ⟩ Used in section 3
⟨ Look at the puzzle that cannot exist 38 ⟩ Used in section 3
⟨ Name an item for every element of every combination 31 ⟩ Used in section 30
⟨ Name the symmetric puzzles, when there are few enough to name 22 ⟩ Used in section 23
⟨ Read the command line 2 ⟩ Used in section 1
⟨ Report the second rows that can occur 14 ⟩ Used in section 11
⟨ Solve some kakuro with answer 430(d)'s construction 36 ⟩ Used in section 3
⟨ Sort the puzzles into equivalence classes 23 ⟩ Used in section 3
⟨ Take the least member of this orbit as its name 21 ⟩ Used in section 20
⟨ Try every repair that changes one clue 39 ⟩ Used in section 38
⟨ Try the four by four of the closing note 37 ⟩ Used in section 36
⟨ Types 4 ⟩ Used in section 1
⟨ Write an option for every digit a cell can take 32 ⟩ Used in section 30
⟨ Write an option to absorb each combination that goes unused 33 ⟩ Used in section 30

Kakuro by the thousand

	Section	Page
Introduction	1	1
The two diagrams	4	3
Counting the fillings	8	6
The patterns themselves	11	8
Equivalent puzzles	15	11
Forced moves and magic blocks	24	15
Answer 430(d)'s exact cover problem	30	21
The puzzle that cannot exist	38	26
Index	40	27